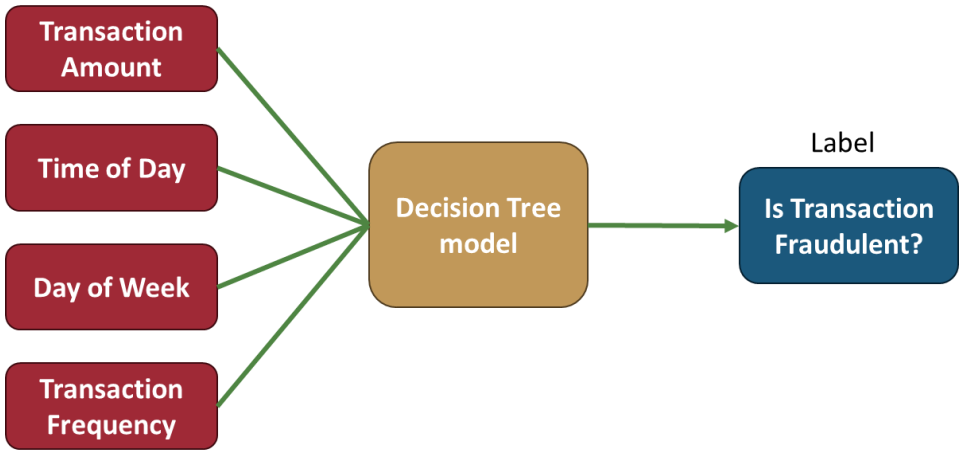
Assessment Brief

Module title	Artificial Intelligence
Module code	COMP2611
Assignment title	Coursework #2 - Decision trees
Assignment type and description	A Programming assignment on decision trees using Python via the Scikit-Learn package.
Rationale	This assignment will help you to have a good understanding of how to work with decision trees and use them for real-world applications.
Tasks to be completed	12 Python programming tasks
Weighting	Marked from 100, worth 20% of the overall module grade.
Submission deadline	March 14th, 5pm
Submission method	Online submission via Gradescope
Feedback provision	Will be delivered via Autograder
Learning outcomes assessed	-Employ artificial intelligence techniques to solve real-world problems, in this instance: decision trees.
Module leader	Arash Rabbani (Co-Lead), Brandon Bennett (Co-Lead)

1. Assignment guidance

This coursework focuses on implementing various tasks related to decision tree classification using the scikit-learn library in Python. The goal is to process and analyze a dataset, train decision tree models, and optimize their performance. We are going to build a model that can detect fraudulent transactions. You will start by reading and filtering the data, splitting it into train and test groups and training a decision tree. Then it will be pruned by increasing the cost complexity parameter. This parameter is then optimized and finally the depths of all trees are compared. The deeper the tree, means it is a more complex model, but it is not necessarily a more accurate one.

The dataset used is a series of transactions and associated features. Each row corresponds to a transaction, and the columns include the transaction amount, time of day, transaction frequency, day of the week, and a binary label indicating whether the transaction is fraudulent (label 1) or not (label 0). Here is the list of dataset headers and description:
transactionAmount: The amount involved in the transaction.
timeOfDay: The time of day when the transaction occurred.
transactionFrequency: How many times a transaction happened in a row.
dayOfWeek: The day of the week when the transaction took place.
labels: Binary labels where 1 indicates a fraudulent transaction and 0 indicates a legitimate one.



You are provided with a template python file that has several functions that need to be completed to carry out the abovementioned decision tree training and testing. In addition, there is a section at the end of the template that starts with **if __name__ == "__main__"** and is called **Main block.** This block demonstrates the usage of functions with the provided dataset. Also, this section is for your local tests to see if the model works reasonably well. For example, it is expected that pruning will maintain or reduce the depth of the model and if that is not the case, it means something is going wrong in your code.

2. Assessment tasks

Download the "CW2.py" and "DT.csv" files from Minerva, under the section for assessment #2. "CW2.py" will act as the template of your code and you should fill out the following functions each of which is presented as a task:

Load the Data (Task 1): [8 marks]
Function: load_data(file_path, delimiter=',')
Description: Load data from a CSV file and return the number of rows, data array, and header list.
Instructions: Provide the file path, specify the delimiter as given. Use pandas to read the csv but the output variable 'data' should be a *numpy* array with rows representing different samples and columns showing different features. 'num_rows' variable is an integer and 'header_list' is a python list containing all the headers.
Filter the data (Task 2): [8 marks]
Function: *filter_data(data)*
Description: Remove rows containing *missing* values from the dataset and give the filtered dataset back as output. The missing values are not actually empty, they have values of "-99".
Instructions: the output should be a *numpy* array.
Data statistics (Task 3): [8 marks]
Function: *statistics_data(data)*
Description: Write a function that calculates the coefficient of variation (standard deviation divided by mean) for each feature in the dataset, excluding the label column.
Instructions: The function must first filter the data using the same approach as Task 2. Return a numpy array containing the coefficient of variations in the same order as the input columns, excluding the label column. Use floating point numbers for the calculation. For any feature with zero mean, return infinity (np.inf) as its coefficient of variation.

Split the data (Task 4): [8 marks]
Function: *split_data(data, test_size=0.3, random_state=1)*
Description: Write a function that splits the dataset into training and testing sets while maintaining label ratios.
Instructions: Use train_test_split of sklearn to make sure that the sampling is stratified, meaning that the ratio between 0 and 1 in the label column stays the same in train and test groups. Keep the *random_state* equal to 1 as it already is.
Train a decision tree (Task 5): [8 marks]
Function: *train_decision_tree(x_train, y_train, ccp_alpha=0)*
Description: write a function that trains a decision tree model with a specified cost complexity parameter.
Instructions: from *sklearn* use *DecisionTreeClassifier* class and its *fit* method. Keep random_state=1 to make the code reproducible.
Make predictions (Task 6): [8 marks]
Function: *make_predictions(model, X_test)*
Description: write a function that takes a trained decision tree model and a set of features (*x_test*) and gives back the predicted label (*y_test_predicted)*.
Evaluate Model (Task 7): [8 marks]
Function: *evaluate_model(model, x, y)*
Description: Write a function that evaluates the performance of the model on the test dataset, returning accuracy and recall.
Optimal Cost Complexity Parameter (Task 8): [8 marks]

Function: *optimal_ccp_alpha(x_train, y_train, x_test, y_test)*
Description: Write a function that returns the most suitable (optimal) cost complexity parameter (ccp_alpha) for pruning a decision tree as a float number. The function should train an initial unpruned model with ccp_alpha set to 0, then search through increasing values of ccp_alpha starting from 0.001. The goal is to find the highest possible ccp_alpha value where the test accuracy stays within ±1% of the unpruned model's accuracy. The function should return the last valid ccp_alpha value before the accuracy drops below the acceptable threshold.
Instructions: The returned ccp_alpha must be within the range [0,1] and returned as a float number. For implementing the search, use a for loop that explores the space from 0.001 upward with fixed increments of 0.001. The search should be limited to 1000 iterations maximum to prevent infinite loops. When the accuracy drops below the -1% threshold compared to the unpruned model, return the last valid ccp_alpha value that maintained acceptable accuracy. If ccp_alpha reaches 1.0 without finding a breaking point, return the last valid value found. In cases where the unpruned model achieves perfect accuracy (1.0), return 0.0 as the optimal value.
Example: Consider an unpruned model with accuracy of 0.85. The acceptable accuracy range would be between 0.84 and 0.86 (±1%). The function should return the highest ccp_alpha value where the model accuracy remains within this range. For instance, if the accuracy drops to 0.83 when ccp_alpha reaches 0.15, the function should return 0.14, which was the last ccp_alpha value that maintained acceptable accuracy.

Decision Tree Depth (Task 9): [8 marks]
Function: *tree_depths(model)*
Description: Write a function that gets a decision tree model and gives back the depth of it. So, in the test stage we can use this function to compare the depth of the initial, pruned, and optimized decision trees.
Feature importance (Task 10): [8 marks]
Function: *important_feature(x_train, y_train, header_list)*
Description: Write a function that identifies the single most important feature for fraud prediction by reducing the decision tree to depth 1 through cost complexity pruning.
Instructions: Starting with ccp_alpha of 0, increase it by increments of 0.01 until the resulting decision tree has exactly depth 1. The feature used for the split at depth 1 is considered the most important. Return the feature name as a string, exactly matching its name in header_list. If multiple features are viable at depth 1, return the one that appears first in header_list. If the tree depth jumps from higher depth directly to 0, return the feature used in the last non-zero depth tree. Keep ccp_alpha within [0, 1].

Single Feature Optimization (Task 11): [10 marks]
Function: optimal_ccp_alpha_single_feature(x_train, y_train, x_test, y_test, header_list)
Description: Write a function that finds the optimal cost complexity parameter when using only the most important feature for prediction. The model should utilize only this single feature but can create multiple decision boundaries using this same feature if doing so improves classification performance.
Instructions: First identify the most important feature using the exact method from Task 10. Create new training and test datasets containing only this feature. Then use your implementation of optimal_ccp_alpha from Task 8 with these reduced datasets to find the optimal parameter value. Return this value as a float number.

Two-Feature Tree Analysis (Task 12): [10 marks]
Function: optimal_depth_two_features(x_train, y_train, x_test, y_test, header_list)

Description: This function determines the optimal tree depth when using exactly two features for classification. The function should first find the most important feature using the method you created in Task

10. Then, after removing this feature from consideration, apply the same method to find the second most important feature. Using only these two features, find the optimal ccp_alpha using your implementation of Task 8. Train a decision tree with this ccp_alpha value and return its depth. If multiple features tie when selecting the second most important feature, use the one that appears first in the header_list. The returned depth must be a positive integer representing the final tree depth. If the optimal tree has multiple possible depths due to equivalent ccp_alpha values, return the smallest valid depth.

3. General guidance and study support

Please refer to the module handbook, teaching slides, and skills@library.

4. Assessment criteria and marking process

Your submission will be marked via AutoGrader and will go through 12 tests to evaluate the 12 tasks. The tests are similar to the function calls in the "main block" of the template that is shared with you (CW2.py) and will fail if values or size of the requested output does not match to the solution.
When you submit your code, you will get example feedback for Tasks 1 to 5 (40 marks). That gives you a good idea showing your code is on the right track. Feedback on the rest of the tasks is reserved for the final marking of the coursework and will not be available during the submission period. For the final marking the used dataset will change so the answers cannot be hard coded or reversed engineered. As a note, the "Main block" of your submission won't be marked. Marking will be done only by calling the "functions" and comparing their output with the solution code. Although tests are designed to evaluate each task independently, errors in prerequisite functions may cause failures in dependent tasks. Tasks 1-5 are not guaranteed to pass in the final grading, if they are passing during the submission stage, as the final tests are more rigorous.

5. Presentation and referencing

Include proper recognition and references for any external sources, such as code snippets, papers or repositories, if used to maintain academic integrity. You can cite the external sources in the form of comments at the end of your submission. There is an example at the end of the template.

6. Submission requirements

This is an individual assessment, and each student should make their own submission. You need to submit only the file named "CW2.py" and put your **name** and **email** as a comment at the top of the file in the designated lines. The submitted file should keep its original name as it is "CW2.py", you can download the template file from Minerva under Assessment #2 section, as well as the required dataset which is a csv file. If the file name be changed, the system will assign a mark 0. Also, if the code takes longer than 1 minute to run on a normal machine, server timeout will lead to a mark 0. So, make sure not to cause an infinite loop in your code.

Please do not use any additional Python packages other than those already imported in the provided code. The required/allowed packages are **pandas**, **numpy**, and **scikit-learn**. (**warnings** and **os** packages are there as well for some housekeeping, keep them as it is).

Please do not change the names of the functions provided in the code, nor their input/output argument name and order. Use the exact function names as specified in the template.

Please do not alter the names of the Python file or any associated files provided in the coursework. Changing name of the functions and files will cause losing the mark for the entire coursework due to failure of tests.

7. Academic misconduct and plagiarism

Leeds students are part of an academic community that shares ideas and develops new ones.

You need to learn how to work with others, how to interpret and present other people's ideas, and how to produce your own independent academic work. It is essential that you can distinguish between other people's work and your own, and correctly acknowledge other people's work.

All students new to the University are expected to complete an online Academic Integrity tutorial and test, and all Leeds students should ensure that they are aware of the principles of Academic integrity.

When you submit work for assessment it is expected that it will meet the University's academic integrity standards.

If you do not understand what these standards are, or how they apply to your work, then please ask the module teaching staff for further guidance.

This assessment is in the red category. Assistive AI tools cannot be used.

By submitting this assignment, you are confirming that the work is a true expression of your own work and ideas and that you have given credit to others where their work has contributed to yours.

8. Assessment/ marking criteria grid

Tasks 1-10 are worth 8 marks each, and tasks 11 and 12 are worth 10 marks each, making a total of 100 marks equivalent to 20% of the module final grade.